



Programação web

Revisão para prova

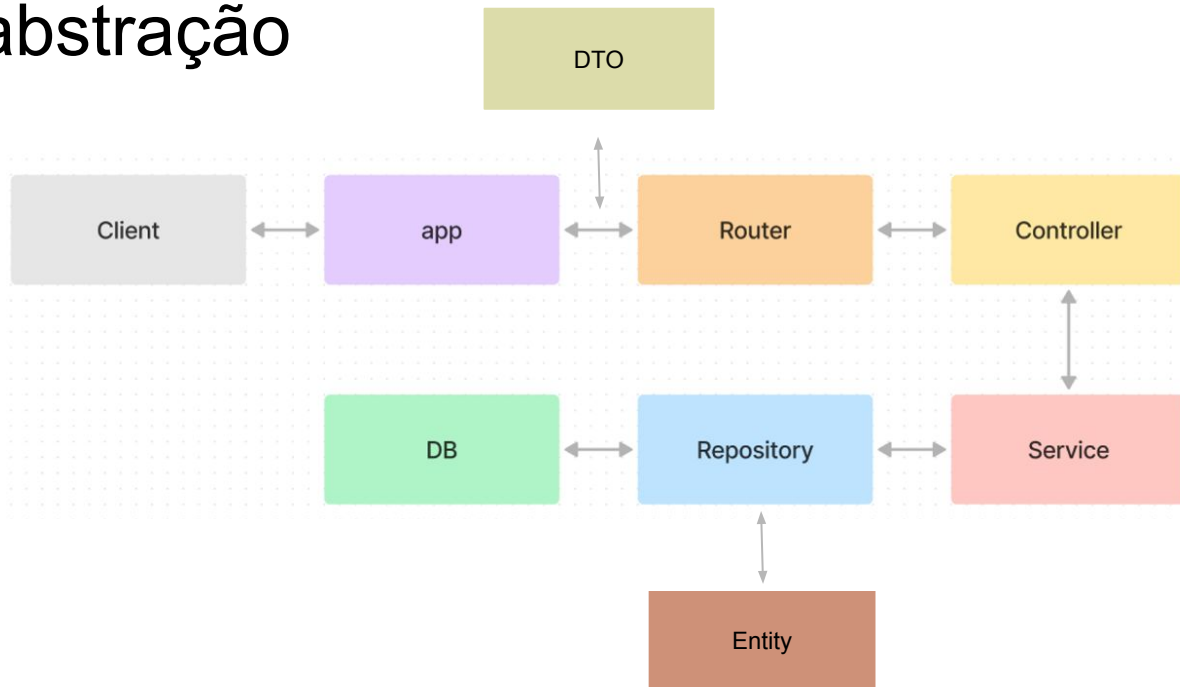
Prof. Diego Cardoso Borda Castro



Relembrando

- Camadas de abstração

- Index/app
- Router
- DTO
- Controller
- Service
- Repositório
- Entidade





Métodos HTTP

app.get("rota", função)	Listar
app.get("rota/:id", função)	Visualizar
app.post("rota", função)	Adicionar
app.put("rota/:id", função)	Atualizar
app.patch("rota/:id", função)	Atualizar
app.delete("rota/:id", função)	Deletar



Adicionando um item

```
1 app.post("/livro", (req, res) => {  
2   const { nome, autor } = req.body;  
3  
4   livros.push({ nome, autor, id: livros.length });  
5  
6   res.status(201).send();  
7 });
```

Método

Rota

Captura do request

Retorno



Pegando um item



```
1  app.get("/livro", (_, res) => {  
2    res.status(200).json(livros);  
3  });  
4  
5  app.get("/livro/:id", (req, res) => {  
6    res.status(200).json(livros[req.params.id]);  
7  });
```

Retorno

Captura do request



Editando um ítem

```
1 app.put("/livro/:id", (req, res) => {  
2   const { nome, autor } = req.body;  
3  
4   livros[req.params.id] = { nome, autor, id: req.params.id };  
5  
6   res.status(200).send();  
7 });  
8
```

Desestruturação



Editando um ítem

- PUT
 - Atualização completa
- PATCH
 - Atualização parcial

```
1  app.patch("/livro/:id", (req, res) => {
2    const { nome } = req.body;
3    const id = req.params.id;
4    const livro = livros[id];
5
6    if (nome)
7      livro.nome = nome;
8
9    res.status(200).send();
10 });
```



Deletando um item



```
1  app.delete("/livro/:id", (req, res) => {  
2    livros.splice(req.params.id, 1);  
3  
4    res.status(200).send();  
5  });
```



Middleware

- Interceptam e processam requisições e respostas no ciclo da API
 - Manipular, modificar ou realizar ações sobre os dados
 - Autenticação e Autorização
 - Mudar formato (JSON)
 - Logging
 - Validação



Middleware

- Parâmetros
 - Req
 - Res
 - Next()

Para um endpoint

```
1 rotas.get('/alunos', logMiddleware, controller.listar);
```

```
1 app.use(logMiddleware)
```

Para todo API

```
1 const logMiddleware = (req, res, next) => {  
2   const { method, url } = req;  
3  
4   console.log(`[${new Date().toISOString()}] ${method} - ${url}`);  
5   next();  
6 };  
7  
8 module.exports = logMiddleware;
```



Typescript

- Linguagem open-source da microsoft
- Superset do javascript
- Tipagem estática



```
1 let aluno = {
2     nome: "Diego",
3     email: "diego.castro@cefet-rj.br",
4     cpf: "11111111111"
5 }
6
7 aluno.cpf = 11111111111;
```



```
class aluno {
    nome: string;
    email: string;
    cpf: string;
}

let novo_aluno: aluno = {
    nome: "Diego",
    email: "diego.castro@cefet-rj.br",
    cpf: 11111111111
}
```



Acesso ao banco

- Manipulação de dados em banco
 - Query pura
 - MySql2
 - Query Builder
 - Maior flexibilidade
 - ORM
 - Organização
 - Entidades

	Gerência	Abstração	Complexidade
Raw	Alto		Baixo
Query builder	Baixo	Baixo	Baixo
ORM	Baixo	Alto	Média



Query builder

- Knex
 - <https://knexjs.org/guide/>
 - `npm i knex --save`
 - `npm i {pg, sqlite3, ...}`
 - `process.env.VAR`

```
1  import Knex from 'knex';
2
3  export const database = Knex( {
4    client: 'postgresql',
5    connection: {
6      host: '127.0.0.1',
7      port : 15432,
8      database: 'postgres',
9      user: 'postgres',
10     password: 'postgres',
11   },
12   pool: {
13     min: 2,
14     max: 10,
15   },
16 });
```



Query builder

- ## Select

- `database('users').select('*')`
- `database('users')`
- `database('users').where('nome', "Diego").first()`
- `knex('users').distinct('unidade')`
- `knex('users').join('profiles', 'users.id', 'profiles.user_id')`
- `knex('orders').count('id as total')`
- `knex('orders').sum('total_price as totalRevenue')`
- `knex.raw('SELECT id, name FROM users WHERE age > ?', [18])`



Query builder

```
const resultado = await knex('matriculas')  
.join('cursos', 'matriculas.curso_id', 'cursos.id')  
.select('cursos.nome as nome_curso')  
.countDistinct('matriculas.aluno_id as total_alunos')  
.groupBy('cursos.nome')  
.orderBy('total_alunos', 'desc')  
.limit(10)  
.offset(0)
```

TABELA PRINCIPAL

JOIN

CAMPOS SELECIONADOS

CONTADOR

AGRUPAMENTO E ORDENAÇÃO

PAGINAÇÃO



Query builder

Insert

```
await knex('alunos').insert([
  { nome: 'Ana', email: 'ana@email.com', curso_id: 2 },
  { nome: 'Carlos', email: 'carlos@email.com', curso_id: 3 }
])

const [id] = await knex('alunos')
  .insert({ nome: 'Lucas', email: 'lucas@email.com', curso_id: 1 })
  .returning('id')

await knex.transaction(async trx => {
  await trx('alunos').insert({ nome: 'Paulo', email: 'paulo@email.com' })
})
```



Query builder

```
await knex('alunos')
  .where('id', 1)
  .update({ nome: 'João Atualizado', email: 'joao@novoemail.com' })

const [alunoAtualizado] = await knex('alunos')
  .where('id', 1)
  .update({ nome: 'Novo Nome' })
  .returning(['id', 'nome'])

await knex.transaction(async trx => {
  await trx('alunos')
    .where('id', 1)
    .update({ nome: 'Dentro da transação' })
})
```

Update



Query builder

```
await knex('alunos')
  .where('id', 1)
  .del()

const [alunoDeletado] = await knex('alunos')
  .where('id', 1)
  .del()
  .returning(['id', 'nome'])

await knex.transaction(async trx => {
  await trx('alunos')
    .where('status', 'inativo')
    .del()
})
```

Delete

```
knex('users').where('votes', '>', 100)

const subquery = knex('users')
  .where('votes', '>', 100)
  .andWhere('status', 'active')
  .orWhere('name', 'John')
  .select('id');

knex('accounts').where('id', 'in', subquery)
```



ORM



- Sequelize
 - <https://sequelize.org/docs/v6/>
 - `npm i sequelize --save`
 - `npm i {pg, sqlite3, ...}`
 - `process.env.VAR`

```
import { Sequelize } from "sequelize-typescript";

const sequelize = new Sequelize({
  database: 'postgres',
  dialect: 'postgres',
  username: 'postgres',
  password: 'postgres',
  host: '127.0.0.1',
  port: 15432,
  models: [__dirname + "/models"],
});

export default sequelize;
```



ORM



- Select

- `const users = await User.findAll();`
- `const user = await User.findOne({ where: { id: 1 } });`

```
const users = await User.findAll({
  where: {
    age: { [Sequelize.Op.gt]: 18 }
  }
});
```

- `[Sequelize.Op.gt]`
- `[Sequelize.Op.gte]`
- `[Sequelize.Op.lt]`
- `[Sequelize.Op.lte]`
- `[Sequelize.Op.eq]`
- `[Sequelize.Op.ne]`
- `[Sequelize.Op.like]`



ORM



```
const { Op } = require('sequelize');
const { Matricula, Curso } = require('./models');
const resultado = await Matricula.findAll({
  attributes: [
    [Sequelize.col('curso.nome'), 'nome_curso'],
    [Sequelize.fn('COUNT', Sequelize.fn('DISTINCT', Sequelize.col('matricula.aluno_id'))), 'total_alunos']
  ],
  include: [
    {
      model: Curso,
      attributes: [],
      required: true,
    }
  ],
  group: ['curso.nome'],
  order: [
    [Sequelize.literal('total_alunos'), 'DESC']
  ],
  limit: 10,
  offset: 0
});
```



ORM



```
const novaMatricula = await Matricula.create({
  aluno_id: 123,
  curso_id: 45,
  data_matricula: new Date()
});

const novasMatriculas = await Matricula.bulkCreate([
  { aluno_id: 123, curso_id: 45 },
  { aluno_id: 124, curso_id: 46 },
  { aluno_id: 125, curso_id: 47 }
]);
```

Insert

```
const [total, registrosAtualizados] = await Matricula.update(
  { curso_id: 99 },
  {
    where: { aluno_id: 123 },
    returning: true
  }
);

await Matricula.update(
  { status: 'inativo' },
  { where: { data_matricula: { [Sequelize.Op.lt]: new Date('2024-01-01') } } }
);

const matricula = await Matricula.findOne({ where: { aluno_id: 123 } });
matricula.curso_id = 99;
await matricula.save();
```

Update



ORM



Delete

```
const matricula = await Matricula.findOne({ where: { aluno_id: 123 } });
matricula.curso_id = 99;
await matricula.save();

const totalDeletados = await Matricula.destroy({
  where: { aluno_id: 123 }
});

const matricula = await Matricula.findOne({ where: { id: 1 } });
if (matricula) {
  await matricula.destroy();
}

Aluno.hasMany(Matricula, { foreignKey: 'aluno_id', onDelete: 'CASCADE' });
```



ORM



● Relacionamentos

Relacionamento	Quem Cria a FK?	FK na Tabela do Modelo	Descrição Prática	Exemplo Real
hasOne	Quem possui	Na tabela do outro modelo	Um registro tem um único associado	Aluno → tem um Endereço Aluno cria a chave estrangeira aluno_id na tabela Endereco
belongsTo	Quem referencia	Na própria tabela	Um registro pertence a outro	Endereço → pertence a um Aluno Endereco cria a chave estrangeira aluno_id na tabela Endereco.
hasMany	Quem possui vários	Na tabela do outro modelo	Um registro tem vários associados	Aluno → tem várias Matrículas Aluno cria a chave estrangeira aluno_id na tabela Matricula.
belongsToMany	Ambos os lados	Na tabela intermediária	Muitos registros estão relacionados entre si	Aluno ↔ Curso (via tabela Matriculas) Aluno e Curso têm uma tabela intermediária matriculas, que cria chaves estrangeiras aluno_id e curso_id.



ORM

- **hasOne + belongsTo**
 - Relacionamento 1:1, com a foreign key ficando no lado do belongsTo

```
Aluno.hasOne(Endereco, { foreignKey: 'aluno_id' });  
Endereco.belongsTo(Aluno, { foreignKey: 'aluno_id' });
```



ORM



- **hasMany + belongsTo**
 - Relacionamento 1:N, onde o lado belongsTo guarda a fk

```
Aluno.hasMany(Matricula, { foreignKey: 'aluno_id' });  
Matricula.belongsTo(Aluno, { foreignKey: 'aluno_id' });
```



ORM



- **belongsToMany em ambos os lados**
 - Relacionamento N:N, com uma tabela extra no meio (definida com through)

```
Aluno.belongsToMany(Curso, { through: 'matriculas' });  
Curso.belongsToMany(Aluno, { through: 'matriculas' });
```



ORM



```
module.exports = (sequelize, DataTypes) => {  
  const Aluno = sequelize.define('Aluno', {  
    nome: DataTypes.STRING,  
  });  
  
  Aluno.hasMany(sequelize.models.Matricula, {  
    foreignKey: 'aluno_id',  
    as: 'matriculas',  
  });  
  
  Aluno.hasOne(sequelize.models.Endereco, {  
    foreignKey: 'aluno_id',  
    as: 'endereco',  
  });  
  
  return Aluno;  
};
```

JS

```
export class Aluno extends Model {  
  @Column(DataType.STRING)  
  nome!: string;  
  
  @HasMany(() => Matricula, { foreignKey: 'aluno_id' })  
  matriculas!: Matricula[];  
  
  @HasOne(() => Endereco, { foreignKey: 'aluno_id' })  
  endereco!: Endereco;  
}
```

TS



Nest

```
export class CreateProductDTO {  
  
  @ApiProperty()  
  @IsNumber()  
  categoryId: number;  
  
  @ApiProperty()  
  @IsString()  
  name: string;  
  
  @ApiProperty()  
  @IsNumber()  
  price: number;  
  
  @ApiProperty()  
  @IsString()  
  image: string;  
}
```

DTO

```
@Module({  
  imports: [ConfigModule.forRoot({  
    envFilePath: ['.env.development.local'],  
  })],  
  TypeOrmModule.forRoot({  
    type: 'postgres',  
    database: process.env.DB_DATABASE,  
    host: process.env.DB_HOST,  
    username: process.env.DB_USERNAME,  
    password: process.env.DB_PASSWORD,  
    port: Number(process.env.DB_PORT),  
    entities: [`${__dirname}/**/*.entity{.js,.ts}`],  
    migrations: [`${__dirname}/migration/{.ts,*.js}`],  
    migrationsRun: true,  
  }),  
  UserModule,  
  ProductModule,  
  CartModule  
],  
  controllers: [],  
  providers: [  
    {  
      provide: APP_GUARD,  
      useClass: RolesGuard,  
    },  
  ],  
})  
export class AppModule {}
```

Module
Geral



Nest

Module

```
@Module({
  imports: [ProductEntity],
  providers: [ProductService],
  controllers: [ProductController],
})
export class ProductModule {}
```

```
@ApiBearerAuth()
@Controller('product')
export class ProductController {
  constructor(private readonly productService: ProductService) {}

  @Get()
  async findAll(): Promise<ReturnProduct[]> {
    return this.productService.findAll();
  }

  @Roles(UserType.Admin)
  @UsePipes(ValidationPipe)
  @Post()
  async createProduct(
    @Body() createProduct: CreateProductDTO,
  ): Promise<ProductEntity> {
    return this.productService.createProduct(createProduct);
  }
}
```

Controller



Nest

```
@Injectable()
export class ProductService {
  constructor(
    @InjectRepository(ProductEntity)
    private readonly productRepository: Repository<ProductEntity>,
  ) {}

  async findAll(): Promise<ProductEntity[]> {
    let findOptions = {};

    const products = await this.productRepository.findAll();
    if (!products || products.length === 0) {
      throw new NotFoundException('Not found products');
    }
    return products;
  }

  async createProduct(createProduct: CreateProductDTO): Promise<ProductEntity> {
    return this.productRepository.create({
      ...createProduct,
    });
  }
}
```

Service



Nest

